

Simple Insurance Application – ASP.NET Core Study Project

I am developing a **Simple Insurance Management Application** for study and learning purposes with my friend. We are beginners in **ASP.NET Core Web API**, so the application should be simple, easy to understand, and follow a clean layered architecture.

1. Technology Requirements

Build the application using:

- ASP.NET Core Web API
- C#
- MySQL
- Dapper ORM
- Stored Procedures
- JWT Authentication
- Swagger
- Dependency Injection
- Repository Pattern
- Business/Service Layer
- Controller Layer

Do **not** use Entity Framework Core.

The main purpose of this project is to understand:

- Database design
- Entities and relationships
- REST APIs
- Controllers
- DTOs
- Repository pattern
- Business/Service layer
- Stored Procedures
- JWT authentication
- Role-based authorization
- CRUD operations
- API-to-database communication

Keep the implementation beginner-friendly. Avoid unnecessary enterprise-level complexity.

2. User and Role Concept

The `Users` table should contain **only authentication/credential-related information**.

The application has four roles:

- Admin
- Employee
- Agent
- Customer

The `Users` table is responsible for:

- Username
- Password
- Role
- Active/Inactive status

Do NOT store customer, agent, or employee business/profile information in the `Users` table.

The detailed information should be stored separately:

- `Customers`
- `Agents`
- `Employees`

The relationship should be:

```
Users
|
+---- Customer
|
+---- Agent
|
+---- Employee
```

`Admin` can exist only in the `Users` table if no separate Admin profile information is required.

3. Main Entities

The application should contain the following major entities.

User

Purpose:

Store authentication and authorization information.

Important fields:

- UserId
- Username
- Password
- Role
- IsActive

Controller:

AuthController

Main APIs:

POST /api/auth/login
POST /api/auth/register

Login should:

1. Validate username/password.
2. Check whether the user is active.
3. Generate JWT token.
4. Include UserId and Role in the JWT claims.
5. Return the token to the client.

4. Customer

The `Customers` table stores the customer's business/profile information.

It should contain information such as:

- CustomerId
- UserId
- CustomerCode
- FirstName
- LastName

- DateOfBirth
- Gender
- Email
- MobileNumber
- Address
- PAN
- Aadhaar
- Occupation
- AnnualIncome
- Nominee information
- RegistrationDate
- IsActive

Controller:

CustomersController

APIs:

```
POST    /api/customers
GET     /api/customers/{customerId}
GET     /api/customers
PUT     /api/customers/{customerId}
DELETE  /api/customers/{customerId}
```

Customer APIs should be protected using JWT.

5. Agent

The `Agents` table stores agent-specific information.

Examples:

- AgentId
- UserId
- AgentCode
- FullName
- Email
- MobileNumber
- LicenseNumber
- Branch
- Designation
- CommissionRate

- TotalCommissionEarned
- DateOfJoining
- IsActive

Controller:

AgentsController

APIs:

```
POST    /api/agents
GET     /api/agents/{agentId}
GET     /api/agents
PUT     /api/agents/{agentId}
DELETE  /api/agents/{agentId}
```

Agent APIs should use role-based authorization where appropriate.

6. Employee

The `Employees` table stores employee-specific information.

Examples:

- EmployeeId
- UserId
- EmployeeCode
- FirstName
- LastName
- Email
- MobileNumber
- Department
- Designation
- DateOfJoining
- Salary
- Address
- IsActive

Controller:

EmployeesController

APIs:

```
POST    /api/employees
GET     /api/employees/{employeeId}
GET     /api/employees
PUT     /api/employees/{employeeId}
DELETE  /api/employees/{employeeId}
```

Use role-based authorization.

For example:

```
Admin -> Create/Update/Delete Employees
Employee -> View permitted employee information
```

7. Policy Master

The **Policy Master** should **NOT** be stored as individual customer policies.

Policy Master represents the insurance products offered by the company.

For example:

```
Life Insurance
Health Insurance
Vehicle Insurance
Home Insurance
Travel Insurance
```

The Policy Master configuration should be maintained as **JSON** for this study project.

Example:

```
{
  "policyTypes": [
    {
      "policyType": "Life",
      "description": "Life Insurance",
      "minimumAmount": 100000,
      "maximumAmount": 10000000,
    }
  ]
}
```

```

    "premiumFrequency": [
      "Monthly",
      "Quarterly",
      "Yearly"
    ],
    "active": true
  },
  {
    "policyType": "Health",
    "description": "Health Insurance",
    "minimumAmount": 50000,
    "maximumAmount": 5000000,
    "premiumFrequency": [
      "Monthly",
      "Yearly"
    ],
    "active": true
  }
]
}

```

Create a suitable service/repository or configuration class to read the Policy Master JSON.

Controller:

```
PolicyMasterController
```

APIs:

```

GET /api/policy-master
GET /api/policy-master/{policyType}

```

The Policy Master is used as the **definition/configuration of available insurance products**.

8. Customer Policies

The `Policies` table represents the **actual insurance policies purchased by customers**.

This is different from Policy Master.

Example:

Policy Master

Life Insurance

Health Insurance

Vehicle Insurance

Customer Policies

POL0001 -> Customer 101 -> Life Insurance

POL0002 -> Customer 102 -> Health Insurance

POL0003 -> Customer 101 -> Vehicle Insurance

The `Policies` table should contain information such as:

- PolicyId
- PolicyNumber
- CustomerId
- AgentId
- EmployeeId
- PolicyType
- PolicyAmount
- IsRenewed

Controller:

PoliciesController

APIs:

```
POST /api/policies
GET  /api/policies/{policyId}
GET  /api/policies
GET  /api/policies/customer/{customerId}
GET  /api/policies/agent/{agentId}
PUT  /api/policies/{policyId}
PUT  /api/policies/{policyId}/renew
```

When creating a policy:

1. Validate CustomerId.
2. Validate AgentId if provided.
3. Validate EmployeeId if provided.
4. Validate PolicyType against Policy Master JSON.

5. Create the customer policy.
6. Generate a unique PolicyNumber.

9. Premium

The `Premiums` table stores **all premium/payment-related information for customer policies**.

It should record each premium payment transaction.

Examples:

- PremiumId
- PolicyId
- CustomerId
- AmountPaid
- PaymentDate
- PaymentMode
- TransactionId
- PaymentFrequency
- PaymentStatus
- Remarks

One policy can have multiple premium payments.

Relationship:

```
Customer
|
+---- Policies
      |
      +---- Premiums
      |
      +---- Premiums
      |
      +---- Premiums
```

Controller:

```
PremiumsController
```

APIs:

```
POST /api/premiums
GET /api/premiums/{premiumId}
GET /api/premiums/policy/{policyId}
GET /api/premiums/customer/{customerId}
PUT /api/premiums/{premiumId}
```

The Premium API should validate that:

```
PolicyId exists
CustomerId exists
Policy belongs to Customer
TransactionId is unique
AmountPaid is greater than zero
```

For study purposes, the payment can be simulated. Do not integrate a real payment gateway.

10. Claims

The `Claims` table stores all information related to insurance claims.

A claim belongs to a customer policy.

Claim information includes:

- ClaimId
- PolicyNumber
- CustomerId
- ClaimDate
- ClaimType
- Reason
- ClaimAmount
- ApprovedAmount
- Status
- Remarks
- SettlementDate

Possible claim statuses:

```
Registered
UnderReview
Approved
```

Rejected
Settled

Controller:

ClaimsController

APIs:

```
POST /api/claims
GET  /api/claims/{claimId}
GET  /api/claims/customer/{customerId}
GET  /api/claims/policy/{policyNumber}
PUT  /api/claims/{claimId}
PUT  /api/claims/{claimId}/status
```

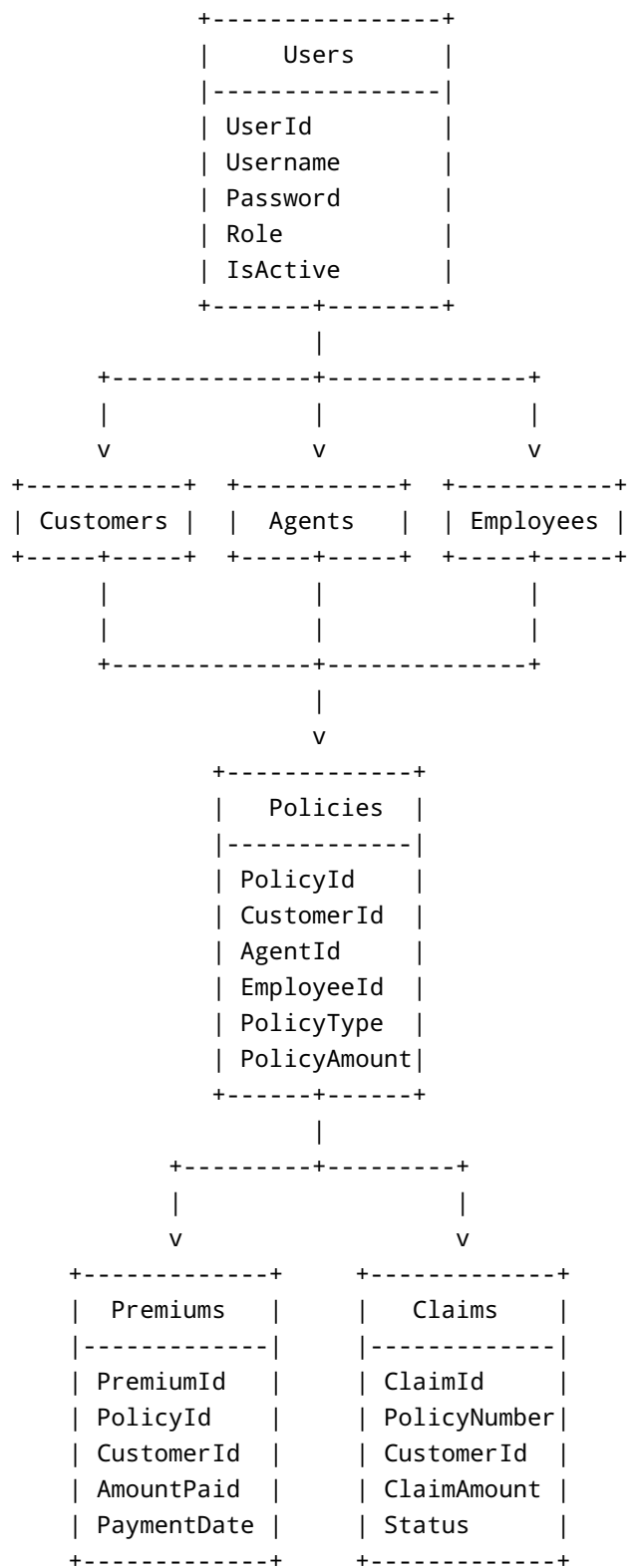
Claim processing example:

```
Customer
  |
  | Submit Claim
  v
Registered
  |
  v
UnderReview
  |
  +-----> Rejected
  |
  v
Approved
  |
  v
Settled
```

The employee should be able to review and update claim status.

11. Overall Entity Relationship

Use the following conceptual relationship:



Policy Master JSON is separate:

```
Policy Master JSON
  |
  v
Available Insurance Products
  |
  v
Policies Table
  |
  v
Customer's Purchased Policy
```

12. Controller List

Create the following controllers:

```
AuthController
CustomersController
AgentsController
EmployeesController
PolicyMasterController
PoliciesController
PremiumsController
ClaimsController
```

Do not create unnecessary controllers.

13. API Responsibilities

Each controller should have a clear responsibility.

Controller	Responsibility
AuthController	Login, registration, JWT
CustomersController	Customer profile management
AgentsController	Agent management
EmployeesController	Employee management
PolicyMasterController	Read available insurance products

Controller	Responsibility
PoliciesController	Customer purchased policies
PremiumsController	Premium/payment transactions
ClaimsController	Claim registration and processing

14. Layered Architecture

Use a simple layered architecture:

```

InsuranceSolution
|
├── Insurance.WebApi
|   ├── Controllers
|   └── Program.cs
|
├── Insurance.Business
|   ├── Interfaces
|   └── Services
|
├── Insurance.Infrastructure
|   ├── Repositories
|   └── Database
|
├── Insurance.Models
|   ├── Entities
|   ├── DTOs
|   └── Requests
|
└── Insurance.Common
    ├── Constants
    ├── Enums
    └── Helpers

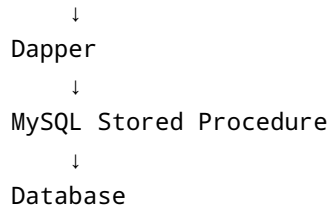
```

Flow:

```

Controller
  ↓
Business/Service
  ↓
Repository

```



15. Important Beginner-Friendly Rules

Keep the project simple.

Do not introduce:

- Microservices
- CQRS
- MediatR
- Event Sourcing
- Repository abstractions with excessive complexity
- Generic repositories unless genuinely useful
- Complex domain-driven design
- Real payment gateway
- Complex insurance calculations

The goal is to help beginners understand how a real ASP.NET Core Web API is structured.

16. Expected Deliverables

First analyze the existing database structure and identify any required improvements.

Then provide:

1. Complete entity list
2. Entity purpose
3. Database table purpose
4. Relationships
5. Controller list
6. API endpoint list
7. HTTP method
8. API purpose
9. Request DTO
10. Response DTO
11. Stored Procedure list

12. Repository methods
13. Business/Service methods
14. Role-based authorization
15. JWT authentication flow
16. Complete project folder structure
17. Recommended database improvements
18. API implementation order

Do not generate the complete code immediately.

First provide the **architecture, entities, database relationships, controllers, APIs, DTOs, and implementation plan.**

After reviewing the design, implement each module step-by-step.